Neural Networks and Reinforcement Learning

Ahmad Assaad

December 2025

1 Introduction

In this report, we will first dive into the theory and algorithms behind neural networks.

2 Neural Networks

2.1 Motivation

The origin of neural networks is the perceptron. The way it works mimics our minds: with sensory input, our minds are able to perceive (the further processing of this input) and output meaningful judgment, such as distinguishing between apples and oranges.

The preliminary question that sparked the development of neural networks was whether we could mimic this process with boolean logic, and hence came the perceptron.

2.2 Architecture

The architecture behind the perceptron is simple: we first have the input layer, in which we assign 0's and 1's based on our data; then we have the hidden layers, where they algebraically transform the data based on sets of weights and biases; and finally we have the output layer, where we obtain 0's and 1's. The goal is to make the perceptron output meaningful information based on the input data we feed it.

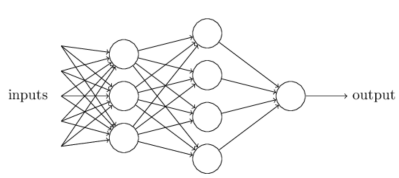

Figure 1: Perceptron Architecture

Now, mathematically, to represent this system we care about how the input data was transformed to eventually reach the output data. To make this representation that much easier, we use vectors and matrices (which to wrap your head around could be thought of lists and lists of lists of numbers, where order is important).

2.3 Notation

We represent all the inputs for the neurons at layer l as a vector where each input is $a_j^{(l)}$:

$$\mathbf{a}^{(l)} = \begin{pmatrix} a_1^{(l)} \\ a_2^{(l)} \\ \vdots \\ a_{n_l}^{(l)} \end{pmatrix}.$$

Similarly, each neuron in layer l (except neurons of the input layer) has connections to all neurons in layer $l-1$. We represent the weights for neurons in layer l as a matrix where the rows represent all the weights for that neuron and the columns represent all the weights coming from a neuron k in layer $l-1$:

$$\mathbf{W}^{(l)} = \begin{pmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1\,n_{l-1}}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \cdots & w_{2\,n_{l-1}}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_l 1}^{(l)} & w_{n_l 2}^{(l)} & \cdots & w_{n_l\,n_{l-1}}^{(l)} \end{pmatrix}$$

Finally, to represent the bias, we use a vector for each layer where the components represent the bias for each neuron k in layer l:

$$\mathbf{b}^{(l)} = \begin{pmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_{n_l}^{(l)} \end{pmatrix}.$$

2.4 How perceptrons work

The magic behind how neural networks work is all in mathematical functions. That is, the activation of one neuron depends on the activation of the previous neuron and the weight between that connection as well as the bias. Remember, in perceptrons activations can only be 0's and 1's and are hence represented by this equation.

However, as you may notice, this highly limits neural networks functions in real-world applications. So, we introduce sigmoid functions in hopes of getting added flexibility not seen in perceptrons.

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

Figure 2: Activation of neuron based on previous activation

2.5 Sigmoid function and neural networks

Now, we introduce the sigmoid function where we get as output real values between 0 and 1 and our input could be any real number.

The sigmoid activation function is defined as

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Perfect, so till now we have defined how networks are structured, how we represent important information throughout it, and finally why we needed this sigmoid function. The question we have yet to answer is how does the learning actually happen? How do we change these weights and biases to get meaningful output?

For that, we introduce gradient descent and backpropagation.

2.6 Gradient Descent

But first, we need to change the problem to something we know how to solve: an optimization problem. For that, we need a function and notion to optimize: a measure of how good or bad our network performs. So, we introduce the cost function, which represents the mean squared error of our network from the actual desired output.

The cost (loss) function of a neural network over a training set of size m is defined as

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{m} \sum_{i=1}^{m} L\Big(\mathbf{a}^{(L)}(x^{(i)}),\, y^{(i)}\Big)\,,$$

where $\mathbf{a}^{(L)}(x^{(i)})$ is the output of the network for input $x^{(i)}$, $y^{(i)}$ is the corresponding target, and $L(\cdot, \cdot)$ is the loss function.

Now, our next step will be moving in the direction that lowers this cost function.

By the linear approximation, the change in the cost is given by the sum of the partial derivatives of the cost with respect to the weights and biases times their corresponding changes (here our variables can be thought of as the weights and biases since they can be changed while the input data cannot):

$$\Delta J \approx \sum_{l} \sum_{j,k} \frac{\partial J}{\partial w_{jk}^{(l)}}\, \Delta w_{jk}^{(l)} \;+\; \sum_{l} \sum_{j} \frac{\partial J}{\partial b_{j}^{(l)}}\, \Delta b_{j}^{(l)}.$$

We therefore think of the cost function as a function whose inputs are all the weights and biases of the network:

$$J = J(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \ldots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}).$$

Updating the weights and biases involves always moving in the direction that lowers the cost function, that is, updating them in the direction opposite to the gradient with a learning rate η:

$$w_{jk}^{(l)} \leftarrow w_{jk}^{(l)} - \eta \frac{\partial J}{\partial w_{jk}^{(l)}}, \qquad b_j^{(l)} \leftarrow b_j^{(l)} - \eta \frac{\partial J}{\partial b_j^{(l)}}.$$

In vector form, this becomes

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} J, \qquad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \nabla_{\mathbf{b}^{(l)}} J.$$

2.7 Backpropagation

2.7.1 error definition

Now, the difficult step is in calculating these partial derivatives to update the weights and biases, and for that we need the backpropagation algorithm. For this, we introduce the idea of an error $\delta^{(l)}$, which quantifies how much the cost changes if the input to a neuron in layer l changes slightly. By the definition of a derivative in calculus, the change in the cost is given by the partial derivative multiplied by the change in the input, and heuristically this error term is just the partial derivative itself:

$$\delta_j^{(l)} \equiv \frac{\partial J}{\partial z_j^{(l)}}.$$

Now, to find the error, which is the partial derivative with respect to the change in input, we use the chain rule. Since we have an explicit expression for the cost function and hence for the output, we can apply the chain rule to define the error again; the error for a neuron becomes the partial derivative of the cost with respect to the output multiplied by how the output changes with respect to the input (the derivative of the sigmoid function):

$$\delta_j^{(l)} = \frac{\partial J}{\partial z_j^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}.$$

For the sigmoid activation function $a_j^{(l)} = \sigma(z_j^{(l)})$, this gives

$$\delta_j^{(l)} = \frac{\partial J}{\partial a_j^{(l)}} \sigma'(z_j^{(l)}).$$

In this way, we successfully transform the problem into variables we have direct access to.

2.7.2 Propagation

Now, in vector form, we define the error in the lth layer in terms of the error in the layer after it. For this, we use the chain rule again. Logically, the error propagates backward: the error in layer l is given by the weights in layer $l+1$ multiplying the error in layer $l+1$, element-wise multiplied by the derivative of the sigmoid function:

$$\boldsymbol{\delta}^{(l)} = \left(\mathbf{W}^{(l+1)}\right)^{\top} \boldsymbol{\delta}^{(l+1)} \odot \sigma'\left(\mathbf{z}^{(l)}\right).$$

Now we have successfully found the error for any neuron in any layer of the network according to the equation above.

Our next step is to find the partial derivatives of the cost J with respect to the weights and biases using these errors, which again follows directly from the chain rule:

$$\frac{\partial J}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}, \qquad \frac{\partial J}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)}\right)^{\top}.$$